

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (Bsc.IT Semester 3)
DSA

Practical 9

Roll No. : 45	Name: Raj Upadhyay
Class : SYBSC IT	Batch : 1st
Date of Assignment : 12/08/2026	Date/Time of Submission : 12/0/82026

PART A - THEORY

1. Linear Search

Definition:

Linear Search is a searching technique in which each element of the array is checked **sequentially from the beginning until the required element is found or the end of the array is reached.**

Example:

Array: 10 25 30 45 60

↑

Search

The elements are checked one by one.

Features:

- Works on both **sorted and unsorted arrays.**
- Simple to implement.
- Suitable for small lists.
- Time Complexity: **O(n).**

2. Binary Search

Definition:

Binary Search is a searching technique that repeatedly **divides a sorted array into two halves** and determines which half may contain the required element.

Important: The array must be **sorted** before applying binary search.

10 20 30 40 50 60 70

↑

Middle

If the search element is smaller than the middle element, search the **left half**.
Otherwise, search the **right half**.

Features:

- Requires a **sorted array**.
- Faster than linear search for large sorted arrays.
- Time Complexity: **$O(\log n)$** .

Comparison

Feature	Linear Search	Binary Search
Array	Sorted/Unsorted	Sorted only
Method	Sequential checking	Divides into halves
Time Complexity	$O(n)$	$O(\log n)$
Simplicity	Easy	Comparatively complex

Suitable for Small/unordered data Large/sorted data

Conclusion

Linear Search checks elements one by one, whereas Binary Search reduces the search area by half at each step. Therefore, **Binary Search is generally faster for large sorted arrays.**

Practical 9A

PART B - QUESTIONS

Searching Algorithms:

Write programs to implement and compare:

a. Linear search

Code :

Linear Search in Python

def linearSearch(array, n, x):

Going through array sequentially

for i in range(0, n):

if array[i] == x:

return i

return -1

array = [2, 4, 0, 1, 9]

x = 3

n = len(array)

```
result = linearSearch(array, n, x)
```

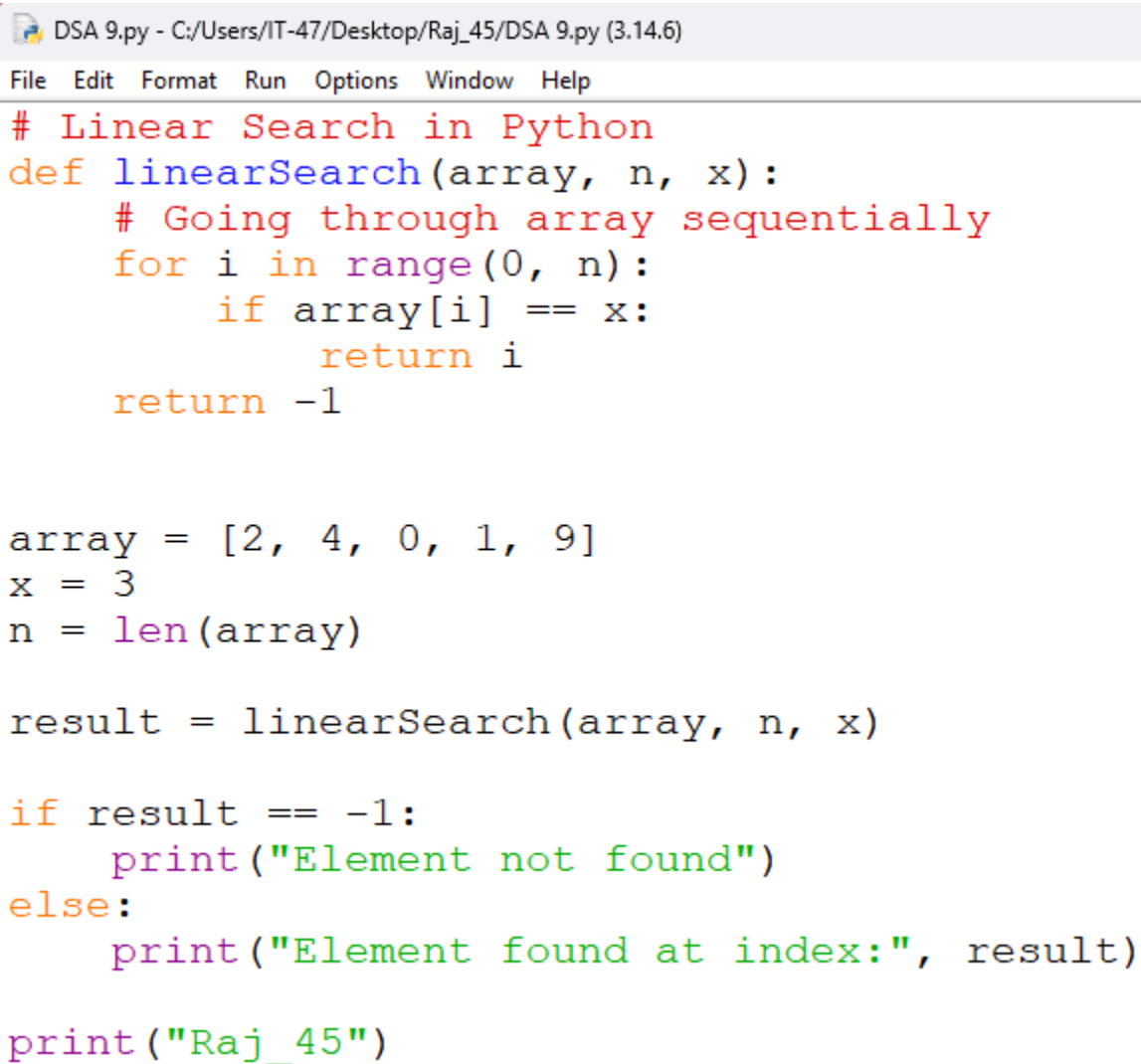
```
if result == -1:
```

```
    print("Element not found")
```

```
else:
```

```
    print("Element found at index:", result)
```

```
print("Raj_45")
```

A screenshot of a Python IDE window titled "DSA 9.py - C:/Users/IT-47/Desktop/Raj_45/DSA 9.py (3.14.6)". The window has a menu bar with "File", "Edit", "Format", "Run", "Options", "Window", and "Help". The code is written in a monospaced font with syntax highlighting. The code defines a function linearSearch, initializes an array and a search value, and prints the result. The code is as follows:

```
# Linear Search in Python
def linearSearch(array, n, x):
    # Going through array sequentially
    for i in range(0, n):
        if array[i] == x:
            return i
    return -1

array = [2, 4, 0, 1, 9]
x = 3
n = len(array)

result = linearSearch(array, n, x)

if result == -1:
    print("Element not found")
else:
    print("Element found at index:", result)

print("Raj_45")
```

Output :

```
===== RESTART: C:/Users/IT-47/Desktop/Raj_45/DSA 9.1
Element found at index: 1
Raj_45
>
===== RESTART: C:/Users/IT-47/Desktop/Raj_45/DSA 9.1
Element not found
Raj_45
>
```

b. Binary search (on a sorted array)

Code :

Binary Search in Python

def binarySearch(array, x, low, high):

Repeat until the pointers low and high meet each other

while low <= high:

mid = low + (high - low) // 2

if x == array[mid]:

return mid

elif x > array[mid]:

low = mid + 1

else:

high = mid - 1

return -1

array = [3, 4, 5, 6, 7, 8, 9]

x = 6

result = binarySearch(array, x, 0, len(array) - 1)

if result != -1:

print("Element is present at index " + str(result))

else:

print("Not found")

print("Raj_45")

```
*DSA 9.py - C:/Users/IT-47/Desktop/Raj_45/DSA 9.py (3.14.6)*
File Edit Format Run Options Window Help
# Binary Search in Python
def binarySearch(array, x, low, high):
    # Repeat until the pointers low and high meet each other
    while low <= high:
        mid = low + (high - low) // 2

        if x == array[mid]:
            return mid

        elif x > array[mid]:
            low = mid + 1

        else:
            high = mid - 1
    return -1
array = [3, 4, 5, 6, 7, 8, 9]
x = 6

result = binarySearch(array, x, 0, len(array) - 1)

if result != -1:
    print("Element is present at index " + str(result))
else:
    print("Not found")
    print("Raj_45")
```

Output :

```
>|===== RESTART: C:/Users/IT-47/Desktop/Raj_45/DSA 9.py =====
|Element is present at index 3
|Raj_45
|<
```

Element is present at index 3
Raj_45

Element is present at index 1
Raj_45

===== RESTA
Element is present at index 6
Raj_45

Not found
Raj_45
